Python Tutorial Notebook

Loops in Python (Beginner-Friendly)

This notebook teaches **loops** step-by-step with examples, flowcharts, common mistakes, and **practice tasks**.

Previously covered topics:

1) `print()` and `input()`
2) Variables and Data Types
3) Conditionals and Strings

In this notebook:

- `for` loop
- `while` loop
- `range()`
- `break` and `continue`
- Simple patterns (counting, summing, checking)

Part A — What is a loop?

A **loop** repeats a block of code multiple times.

Loops are useful when you want to:

- Repeat printing something
- Count numbers
- Sum values
- Repeat input until a condition is met

Two main types of loops in Python (basic level)

1. `for` loop (repeat over a sequence / range of numbers)
2. `while` loop (repeat while a condition is True)

Part B — `for` loop

Theory

A `for` loop repeats a block for each item in a sequence.

At beginner level, we mostly use it with `range()` to repeat a fixed number of times.

Syntax

```
for variable in sequence:
    # block
```

Using `range()` (very common)

`range(start, stop, step)` generates numbers:

- `start` is optional (default 0)
- `stop` is not included (important!)
- `step` is optional (default 1)

In [1]:

```
# range examples
print(list(range(5)))         # 0,1,2,3,4
print(list(range(2, 6)))      # 2,3,4,5
print(list(range(1, 10, 2)))  # 1,3,5,7,9
```

Note: `list(range(...))` is used here just to show the numbers.

In real beginner code, you typically loop directly over `range()`.

In [2]:

```
# Example 1: Print 1 to 5
for i in range(1, 6):
    print(i)
```

In [3]:

```
# Example 2: Print a message 3 times
for _ in range(3):
    print("Hello!")
```

Looping through a string

A string is a sequence of characters, so you can loop through it.

In [4]:

```
word = "PYTHON"
for ch in word:
    print(ch)
```

Part C — `while` loop

Theory

A `while` loop repeats **as long as** a condition is True.

Syntax

```
while condition:
    # block
```

Be careful: if the condition never becomes False, it becomes an **infinite loop**.

In [5]:

```
# Example 1: Print 1 to 5 using while
i = 1
while i <= 5:
```

```
    print(i)
    i += 1
```

In [6]:

```
# Example 2: Keep asking until user enters 'yes'
answer = input("Type yes to continue: ").strip().lower()

while answer != "yes":
    answer = input("Try again. Type yes: ").strip().lower()

print("Good!")
```

Part D — `break` and `continue`

`break` *(Theory)*

Stops the loop immediately.

`continue` *(Theory)*

Skips the rest of the current iteration and goes to the next one.

In [7]:

```
# break example: stop when i becomes 4
for i in range(1, 10):
    if i == 4:
        break
    print(i)
```

In [8]:

```
# continue example: skip even numbers
for i in range(1, 11):
    if i % 2 == 0:
        continue
    print(i)
```

Part E — Common loop patterns (beginner)

1) Counter pattern

Count how many times something happens.

In [9]:

```
text = "banana"
count_a = 0

for ch in text:
    if ch == "a":
        count_a += 1

print("Number of a:", count_a)
```

2) Accumulator pattern (sum)

Add values step-by-step.

In [10]:

```
# Sum of 1 to 5
total = 0
for i in range(1, 6):
    total += i
print("Sum:", total)
```

Part F — Flowcharts (ASCII)

`for` *loop (idea)*

```
Start
  |
Get next value from sequence
  |
Is there a value?
 / Yes  No
 |    |
Run  End
block
 |
Back to get next value
```

`while` *loop (idea)*

```
Start
  |
Condition?
 / Yes  No
 |    |
Run  End
block
 |
Back to condition
```

Part G — Common mistakes (avoid these)

1) **Forgetting indentation**
2) **Infinite while loop** (forgetting to update the variable)
3) `range(stop)` stops at **stop-1**
4) Using `break` too early (loop ends sooner than expected)

In [11]:

```
# Infinite loop example (DON'T RUN):
# i = 1
# while i <= 5:
#     print(i)
```

```
#     # i is never updated -> infinite loop
```

Practice Tasks (Loops + Conditionals + Strings)

■ Allowed: `print`, `input`, variables, strings, conditionals, loops (`for`/`while`), `range`, basic math

■ Avoid: lists, functions, recursion, files, advanced modules

Write your solution in the empty cell under each task.

Task 1 — Print 1 to 10

Use a `for` loop to print numbers from 1 to 10 (each on a new line).

In [12]:

```
# Write your solution here
```

Task 2 — Print Even Numbers

Take an integer `n` from the user and print all even numbers from 1 to `n`.

In [13]:

```
# Write your solution here
```

Task 3 — Sum of 1 to n

Take `n` from the user and calculate the sum: 1 + 2 + ... + n. Print the sum.

In [14]:

```
# Write your solution here
```

Task 4 — Multiplication Table

Take a number `x` and print its table from 1 to 10.

Example: 5 x 1 = 5 ... 5 x 10 = 50

In [15]:

```
# Write your solution here
```

Task 5 — Factorial (basic)

Take `n` and compute `n!` using a loop.

Example: 5! = 120

In [16]:

```
# Write your solution here
```

Task 6 — Count Vowels

Take a string and count vowels (a, e, i, o, u). Print the count.

Hint: use `.lower()` and `for ch in text:`

In [17]:

```
# Write your solution here
```

Task 7 — Count Digits in a String

Take a string and count how many characters are digits.

Hint: `ch.isdigit()`

In [18]:

```
# Write your solution here
```

Task 8 — Reverse a String Using Loop

Take a word and reverse it using a loop (not using slicing).

Hint: build a new string step-by-step.

In [19]:

```
# Write your solution here
```

Task 9 — Password Check (while)

Ask user to enter a password. Keep asking until they enter `python123`.

Then print `Access granted`.

In [20]:

```
# Write your solution here
```

Task 10 — Limited Attempts (while)

Ask user to enter a PIN. Give only 3 attempts.

Correct PIN: 4321.

If correct, print `Unlocked`. Otherwise after 3 wrong attempts print `Blocked`.

In [21]:

```
# Write your solution here
```

Task 11 — Find First 'a' Position

Take a word and find the index (position) of the first 'a' (or 'A') using a loop.

If not found, print `Not found`.

Hint: use a counter variable for index.

In [22]:

```
# Write your solution here
```

Task 12 — Nested Loop Pattern

Print this pattern using nested loops (n = 5):

*

*

*

In [23]:

```
# Write your solution here
```

Small Mini-Project (Optional, still beginner-friendly)

Number Guess (very simple):

- Set a secret number (example: 7)
- Ask user to guess
- If guess is too high → print `Too high`
- If too low → print `Too low`
- Keep looping until correct → print `Correct!`

In [24]:

```
# Write your solution here
```